Arrays and Graphs

Mitchell Gerrard

October 20, 2020

1 1 What would go wrong if fabs were omitted?

well the term could be negative and the total positive at the same time so we would lose terms which would be significant to us .

2 2 Why do you think the number 1.0e-7 is chosen for the stopping condition, at least in comparison to a much larger or much smaller number, say 1e-3, or 1e-26?

If we go to small we waste time with computation on extremely small terms which we cant process well in python and if it is to large are series does not converge on the function we wish to approximate

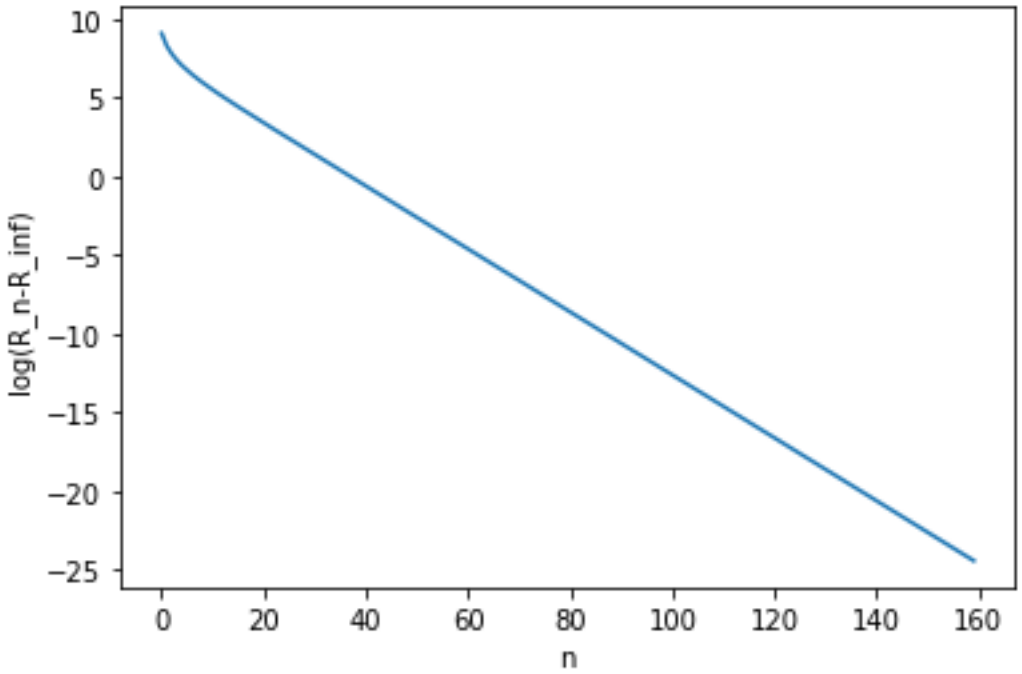
3 3 Create your own script and copy the g(x) function into it. Your script should then make suitable arrays of values of x and g(x) and plot a graph of g(x) for $0 \leq x \leq 4\pi$. Make sure your script saves the plot to a png file (do not use show()).

script plotting.py does the Fourier series expansion of our function from 0 to 4π and then plot the results and save it a g(x).png

4 4 Write a program to compute R_∞ and print the result. It may be helpful to consider how we used a while loop in the g(x) function earlier.

script rinf.py runs until the term is 10^7 less than the sum of the terms to that point giving us a large enough n.

5 5 From your graph explain why when n is large R_n is given approximately by: $R_n = R_\infty + Ae^{-Bn}$



if you look at the graph for large n we have a straight line so $ln(R_n - R_\infty = -bn + c$ with b being a positive constant an c being a constant so rearranging we have

$$R_n - R_\infty = e^{-bn+c} = Ae^{-bn}$$

$$R_n = R_\infty + Ae^{-bn}$$

6 bonus work

6.1 libraries

- svgpathtools
- math
- svg.path
- matplotlib.pyplot
- numpy

6.2 theory

A Fourier series is a periodic function made with summation of sine and cosines. what we need is the complex version of this to fit to a imaginary function so we can plot it on a 2d plane.
Because complex numbers can be expressed in polar form and then can be put in terms of sines and cosines we are going use polar form in this way. we can use a sum of our exponentials to make our function like this like this:

$$f(t) = \sum_{n=-\infty}^{\infty} c_n e^{2\pi nit}$$

for our purposes we know the function but need to find our tuning knobs c_n we derive it like this

$$\int_0^1 f(t)dt = \int_0^1 (\sum_{n=-\infty}^{\infty} c_n e^{2\pi nit})dt$$

doing this next step is not mathematically rigorous but to much to go in to right now yo need real analysis for this but we can distribute the integral.

$$\int_0^1 f(t)dt = ... \int_0^1 c_-1 e^{2\pi -1it}dt + \int_0^1 c_0 e^{2\pi 0it}dt + \int_0^1 c_1 e^{2\pi 1it}dt...$$

this gives us averages of the individual functions we get c_0 because everything else will average to 0. and we can get the rest by multiplying through by

$e^{-2ni\pi t}$ meaning that we get

$$c_n = \int_0^1 f(t)e^{-2ni\pi t}dt$$

then finding the c_ns we can then.

$$f(t) = \sum_{n=-\infty}^{\infty} c_n e^{2\pi nit}$$

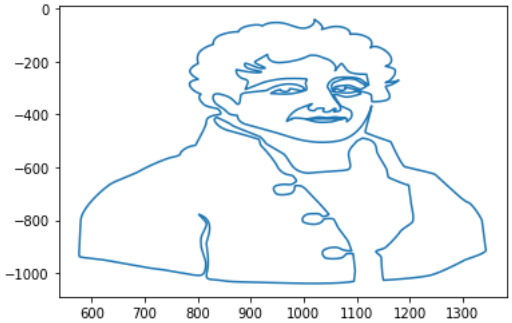
finsing every point you want to find.

6.3 applications

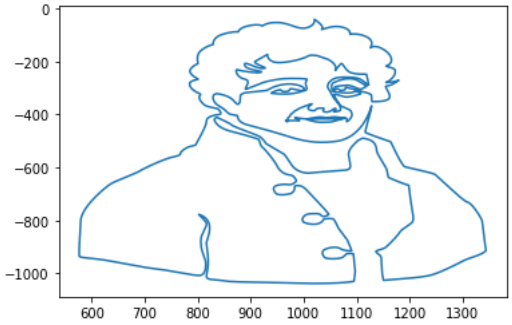
so from an svg file we need to find point evey t with a difference dt. because we cannot have a fully defined complex functiopn fom an svgh we get the best we can then we pick how many comples numbers we want in our sum then we do $c_n = \int_0^1 f(t)e^{-2ni\pi t}dt$ for all the complex numbers we have. and we need to do this numerical because we do not have a full function so we do then we put them in to our sum $f(t) = \sum_{n=-k}^{k} c_n e^{2\pi nit}$ with k being half the numebr of complex numbers we found and then do that for all t for our function.

6.4 results

the svg file



mathematically produced.


0
−200
−400
−600
−800
−1000
600
700
800
900
1000
1100
1200
1300